Tertiary Infotech Academy Pte Ltd
UEN: 201200696W

LEARNER GUIDE

For

Designing and Implementing Microsoft DevOps Solutions (AZ-400)

TGS Ref No: TGS-2024042605

Conducted by
Tertiary Infotech Academy Pte Ltd
UEN: 201200696W
Version 1

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Document Version Control Record

Version Number	Effective Date of Release	Summary of Included Changes	Author
1	8 July 2026	First AZ-400 courseware build aligned to all 8 hands-on labs.	Tertiary Infotech Academy Pte Ltd

Course Information

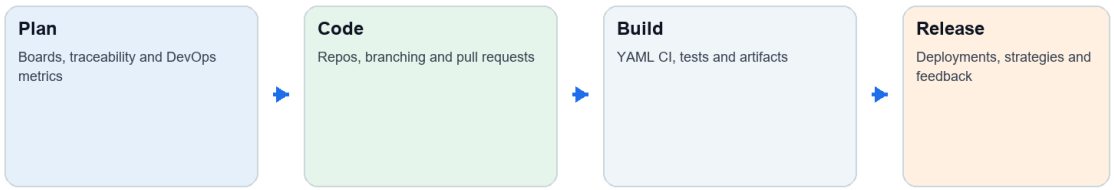
Course Title	Designing and Implementing Microsoft DevOps Solutions (AZ-400)
TGS Ref No.	TGS-2024042605
Duration	2 days / 16 training hours
Delivery Mode	Instructor-led classroom training with hands-on Azure DevOps labs
Lab Source	courseware/labs-source, copied from the referenced GitHub labs repository
Learner Guide Markdown Mirror	LG-Designing and Implementing Microsoft DevOps Solutions (AZ-400).md

Course Overview

This guide supports learners through the complete AZ-400 lab sequence. Each lab section mirrors the source lab with objectives, detailed steps, validation checks, review questions and evidence expectations.

DevOps Delivery Flow

AZ-400 DevOps Delivery Flow



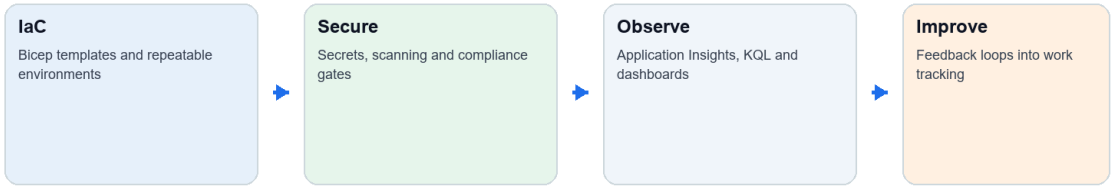
Single source: 8 lab files drive the PPT, Learner Guide, Lesson Plan and Markdown mirror.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The lab sequence follows a complete plan-code-build-release-observe-improve workflow.

Secure DevOps Governance Loop

Secure DevOps Governance Loop

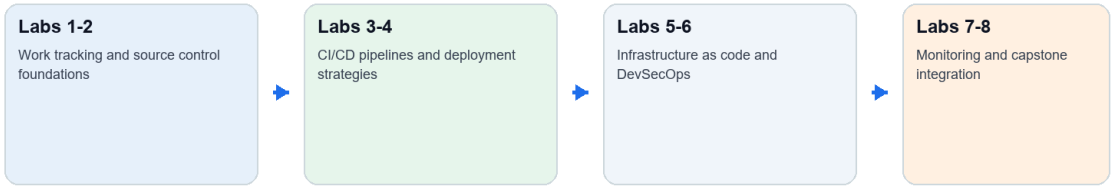


Single source: 8 lab files drive the PPT, Learner Guide, Lesson Plan and Markdown mirror.

Security, compliance, monitoring and feedback are woven through the later labs.

Lab-to-Artifact Alignment

Lab-to-Artifact Alignment Map



Single source: 8 lab files drive the PPT, Learner Guide, Lesson Plan and Markdown mirror.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Each lab produces evidence that is reused in the Learner Guide, slide deck and Lesson Plan.

Visual Lab Roadmap

Lab	Focus	Build / Configure	Evidence
01	DevOps Processes, Boards, and Traceability	Boards → Branch → Commit → Pull Request → Dashboard	Work items follow a clear hierarchy.
02	Source Control, Branching, and Pull Requests	Repository → Branch → Pull Request → Review	Branch strategy fits the team scenario.
03	YAML CI Pipelines, Testing, and Artifacts	YAML Pipeline → Build → Test → Artifact	Pipeline triggers correctly.
04	Release Pipelines and Deployment Strategies	Artifact → Environment → Approval → Deployment	Deployment strategy matches risk and downtime needs.
05	Infrastructure as Code with Bicep and Templates	Bicep Template → Validation → Deployment	Template is parameterized.
06	DevSecOps, Secrets, and Compliance Scanning	Secrets → Scanning → Compliance Gate	No secrets are stored in code.
07	Instrumentation, Monitoring, KQL, and Feedback	Telemetry → KQL → Dashboard → Feedback	Metrics connect to operational decisions.
08	AZ-400 Capstone: End-to-End DevOps Solution	Plan → Code → Build → Release → Monitor	Solution includes traceability.

Lab Rules

1. Do not commit real secrets.
2. Use sandbox subscriptions where possible.
3. Record pipeline run links or screenshots.
4. Delete billable resources after labs.
5. Document design tradeoffs, not only commands.

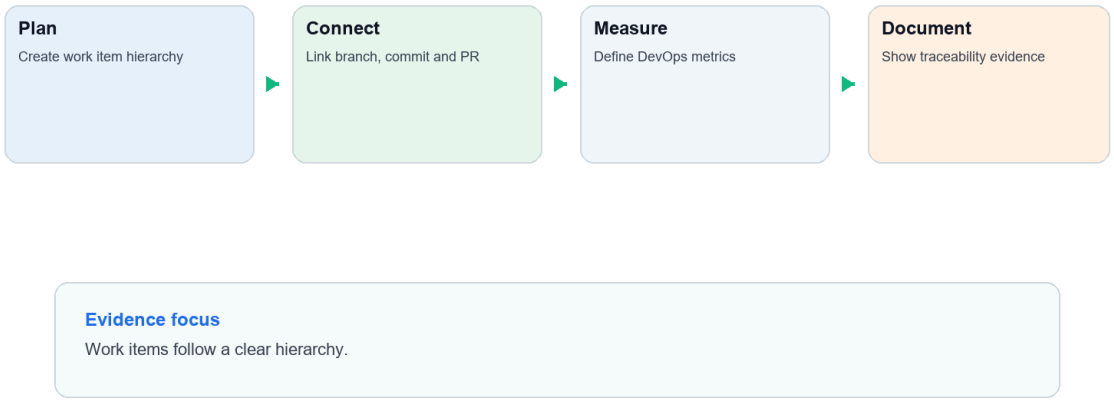
Lab 01 - DevOps Processes, Boards, and Traceability

Source Lab	courseware/labs-source/lab-01-devops-processes-boards-traceability.md

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 01: DevOps Processes, Boards, and Traceability

Visual workflow map



Visual workflow map for Lab 01.

What you will produce: Boards → Branch → Commit → Pull Request → Dashboard. Keep screenshots, links or generated artifacts as evidence.

Objectives

- Create a work tracking structure.
- Link work to code and releases.
- Define DevOps metrics.
- Create project documentation.

Step-by-Step Guide

6. Create or open an Azure DevOps project or GitHub repository.
7. Create sample epics, features, user stories, tasks, and bugs.
8. Define workflow states.
9. Add labels for priority, security, quality, and release.
10. Link a work item to a branch, commit, and pull request.
11. Create a dashboard or metric table.
12. Define lead time, cycle time, deployment frequency, change failure rate, and MTTR.
13. Create Markdown documentation.
14. Add a Mermaid flow diagram.
15. Document traceability from requirement to deployment.

Validation

- Work items follow a clear hierarchy.
- Traceability includes work, code, pipeline, and release evidence.
- Metrics are defined.
- Documentation is readable.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Evidence checkpoint: Work items follow a clear hierarchy.

Review Questions

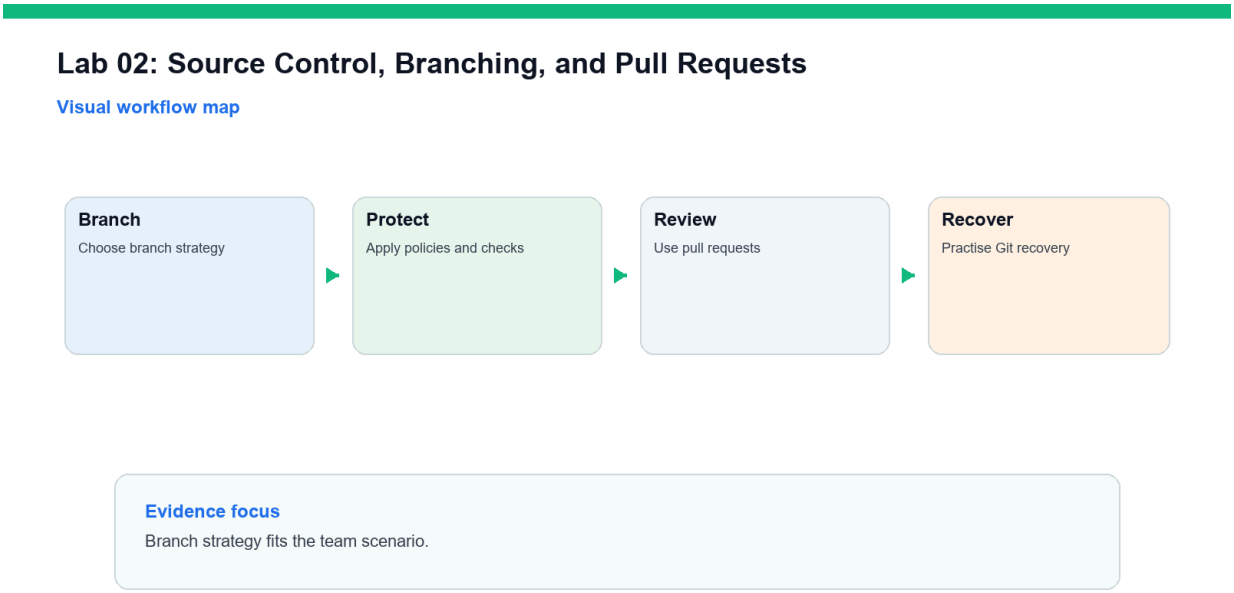
16. Why does traceability matter?
17. What is the difference between lead time and cycle time?
18. How can dashboards improve feedback?
19. Why document architecture and releases?

Learner Evidence

- Screenshot or link to the completed work item, branch, pull request, pipeline run, deployment, dashboard or capstone evidence as relevant.

Lab 02 - Source Control, Branching, and Pull Requests

Source Lab	courseware/labs-source/lab-02-source-control-branching-pull-requests.md



Visual workflow map for Lab 02.

What you will produce: Repository → Branch → Pull Request → Review. Keep screenshots, links or generated artifacts as evidence.

Objectives

- Choose a branch strategy.
- Configure pull request workflow.
- Apply branch protection or policies.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Practise Git recovery.

Step-by-Step Guide

20. Create or open a Git repository.
21. Compare trunk-based, feature branch, and release branch strategies.
22. Choose a branch strategy and document why.
23. Create a feature branch.
24. Make a small change.
25. Commit and push the branch.
26. Open a pull request.
27. Require review and status checks where available.
28. Add a release tag.
29. Recover a previous file version with Git.
30. Document how sensitive data removal should be escalated.

Validation

- Branch strategy fits the team scenario.
- Pull request includes review and checks.
- Tagging convention is documented.
- Git recovery step is successful.

Evidence checkpoint: Branch strategy fits the team scenario.

Review Questions

31. When is trunk-based development useful?
32. Why protect main?
33. What is the purpose of release tags?
34. Why is secret removal from Git history sensitive?

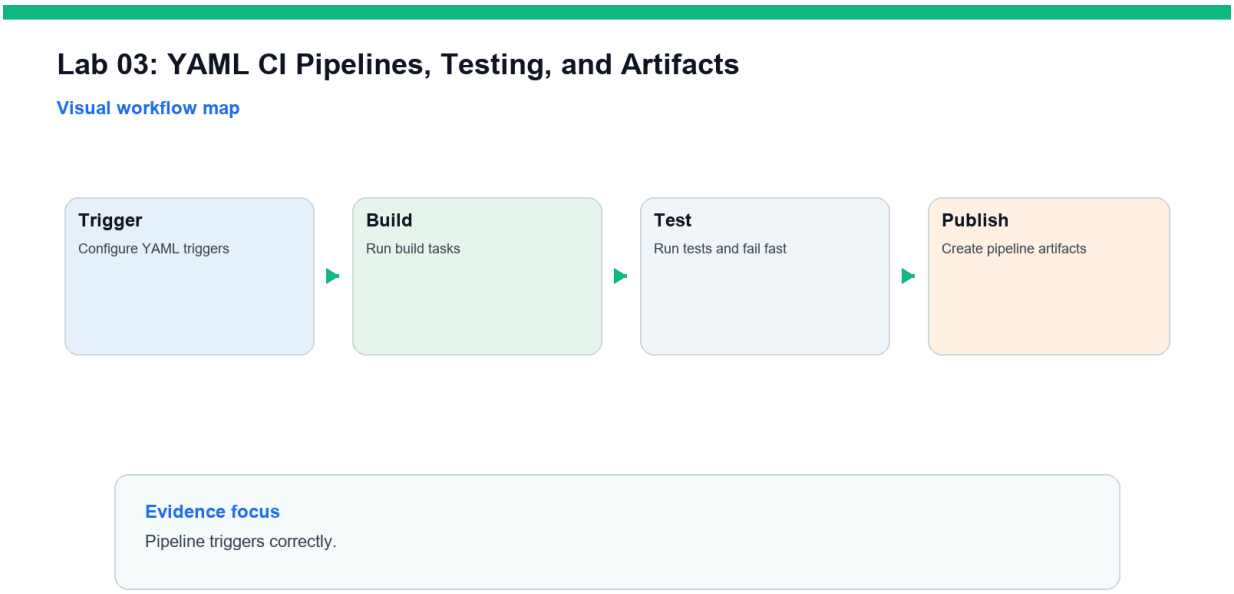
Learner Evidence

- Screenshot or link to the completed work item, branch, pull request, pipeline run, deployment, dashboard or capstone evidence as relevant.

Lab 03 - YAML CI Pipelines, Testing, and Artifacts

Source Lab	courseware/labs-source/lab-03-yaml-ci-pipelines-testing-artifacts.md

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



Visual workflow map for Lab 03.

What you will produce: YAML Pipeline → Build → Test → Artifact. Keep screenshots, links or generated artifacts as evidence.

Objectives

- Create a YAML CI pipeline.
- Configure triggers.
- Run tests.
- Publish artifacts.

Step-by-Step Guide

35. Use a sample application repository.
36. Add a YAML pipeline file.
37. Configure triggers for main and pull requests.
38. Add build steps.
39. Add a unit test step.
40. Publish test results where supported.
41. Publish a build artifact.
42. Use variables or reusable steps.
43. Run the pipeline.
44. Inspect logs.
45. Fix one controlled failure.
46. Record duration and improvement ideas.

Validation

- Pipeline triggers correctly.
- Tests run.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Artifact is published.
- Failure is documented and fixed.

Evidence checkpoint: Pipeline triggers correctly.

Review Questions

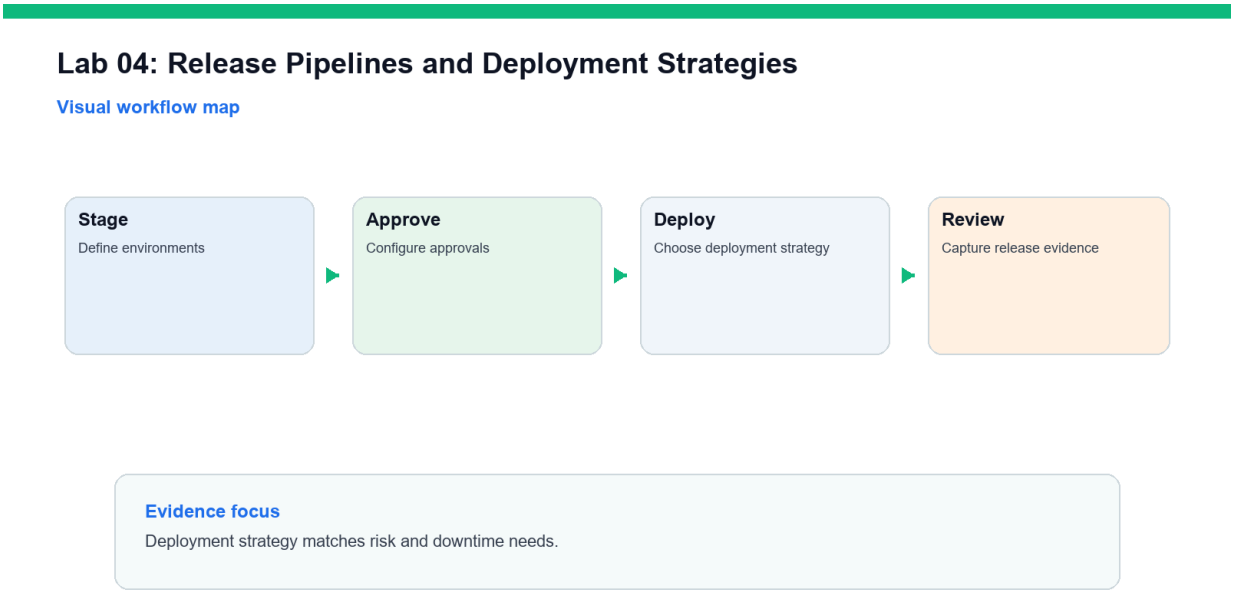
47. Why define pipelines as YAML?
48. What should happen on pull request builds?
49. Why publish artifacts?
50. What makes a test flaky?

Learner Evidence

- Screenshot or link to the completed work item, branch, pull request, pipeline run, deployment, dashboard or capstone evidence as relevant.

Lab 04 - Release Pipelines and Deployment Strategies

Source Lab	courseware/labs-source/lab-04-release-pipelines-deployment-strategies.md



Visual workflow map for Lab 04.

What you will produce: Artifact → Environment → Approval → Deployment. Keep screenshots, links or generated artifacts as evidence.

Objectives

- Design release stages.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Configure environments and approvals.
- Compare deployment strategies.
- Plan rollback and hotfix paths.

Step-by-Step Guide

51. Define dev, test, staging, and production environments.
52. Add deployment stages to a pipeline.
53. Configure approvals or checks for higher environments.
54. Compare blue-green, canary, ring, rolling, feature flags, and deployment slots.
55. Choose a strategy for a sample app.
56. Deploy a sample artifact or simulate deployment.
57. Define rollback criteria.
58. Define rollback steps.
59. Define a hotfix path.
60. Create release notes from commits or PRs.

Validation

- Deployment strategy matches risk and downtime needs.
- Approval/check evidence is captured.
- Rollback path is clear.
- Release notes are generated or drafted.

Evidence checkpoint: Deployment strategy matches risk and downtime needs.

Review Questions

61. When would you use canary deployment?
62. Why are approvals useful?
63. What should a rollback plan include?
64. How do feature flags reduce deployment risk?

Learner Evidence

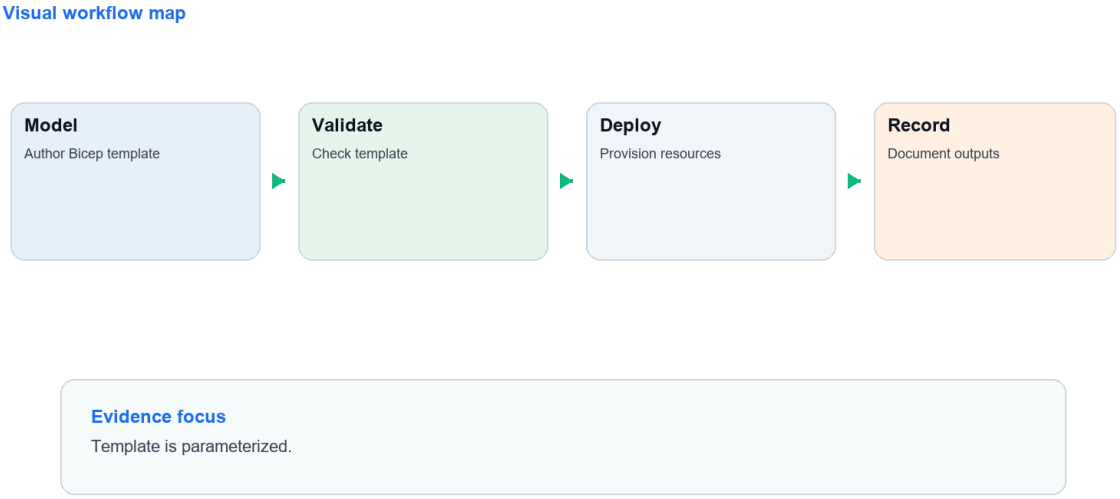
- Screenshot or link to the completed work item, branch, pull request, pipeline run, deployment, dashboard or capstone evidence as relevant.

Lab 05 - Infrastructure as Code with Bicep and Templates

Source Lab	courseware/labs-source/lab-05-infrastructure-as-code-bicep-templates.md

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 05: Infrastructure as Code with Bicep and Templates



Visual workflow map for Lab 05.

What you will produce: Bicep Template → Validation → Deployment. Keep screenshots, links or generated artifacts as evidence.

Objectives

- Define IaC strategy.
- Create a Bicep or template file.
- Validate deployment.
- Document review and drift controls.

Step-by-Step Guide

65. Identify infrastructure required by the sample app.
66. Create or review a Bicep template.
67. Add parameters for environment and location.
68. Add resource tags.
69. Validate the template.
70. Deploy to a sandbox or review simulated deployment output.
71. Store the template in source control.
72. Add an IaC step to a pipeline.
73. Document how drift can occur.
74. Define review and approval rules for IaC changes.

Validation

- Template is parameterized.
- Tags are included.
- Validation or deployment evidence is captured.
- Drift and review controls are documented.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Evidence checkpoint: Template is parameterized.

Review Questions

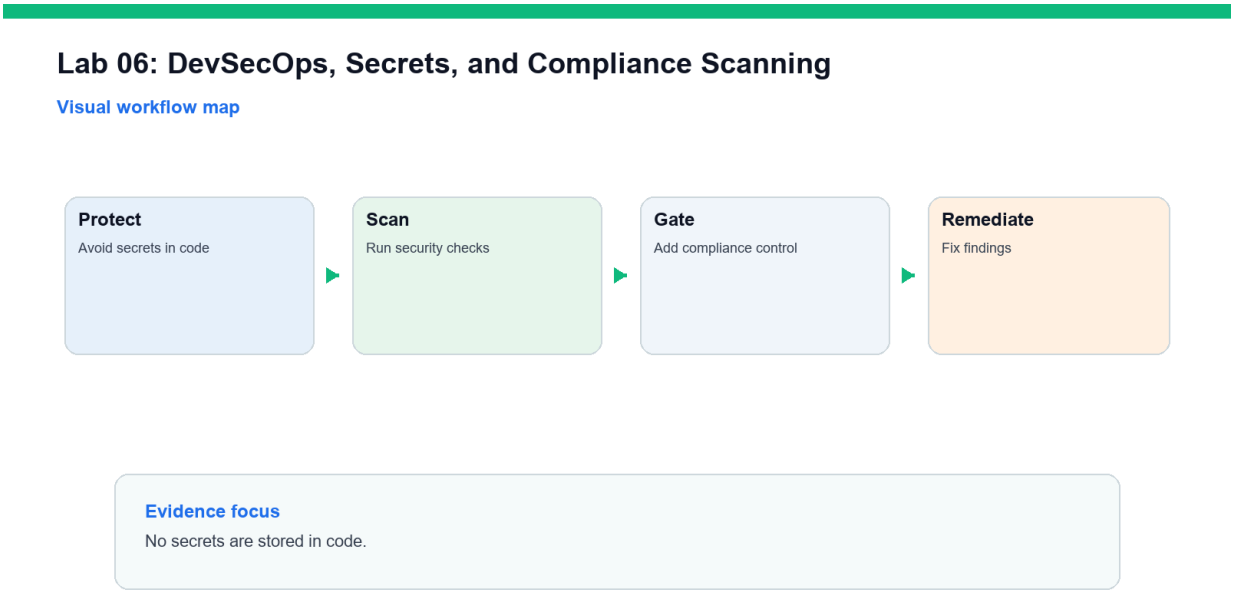
75. Why use IaC?
76. What should be parameterized?
77. What is configuration drift?
78. Why should infrastructure changes use pull requests?

Learner Evidence

- Screenshot or link to the completed work item, branch, pull request, pipeline run, deployment, dashboard or capstone evidence as relevant.

Lab 06 - DevSecOps, Secrets, and Compliance Scanning

Source Lab	courseware/labs-source/lab-06-devsecops-secrets-compliance-scanning.md



Visual workflow map for Lab 06.

What you will produce: Secrets → Scanning → Compliance Gate. Keep screenshots, links or generated artifacts as evidence.

Objectives

- Manage secrets safely.
- Design secure pipeline authentication.
- Configure scanning checks.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Document compliance evidence.

Step-by-Step Guide

79. Identify secrets used by the sample app or pipeline.
80. Move secrets to GitHub secrets, Azure Pipelines variables, or Key Vault.
81. Compare service principals, managed identities, and OIDC.
82. Review service connections or GitHub Actions authentication.
83. Enable dependency scanning where available.
84. Enable secret scanning or add a secret scanning step.
85. Add CodeQL or static analysis conceptually or practically.
86. Review container scanning concepts.
87. Define a policy for critical findings.
88. Create a compliance evidence table.

Validation

- No secrets are stored in code.
- Authentication design follows least privilege.
- Scanning plan covers code, dependencies, secrets, and containers.
- Evidence table is audit-friendly.

Evidence checkpoint: No secrets are stored in code.

Review Questions

89. Why avoid long-lived secrets?
90. What is OIDC workload identity federation?
91. Why scan dependencies?
92. What should block a release?

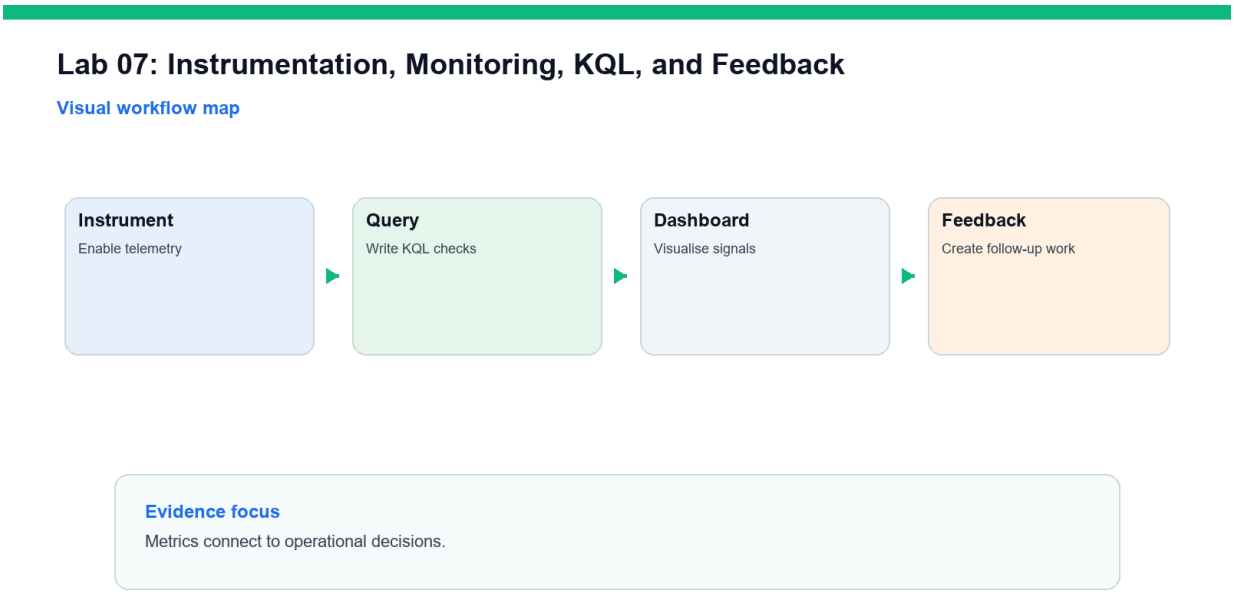
Learner Evidence

- Screenshot or link to the completed work item, branch, pull request, pipeline run, deployment, dashboard or capstone evidence as relevant.

Lab 07 - Instrumentation, Monitoring, KQL, and Feedback

Source Lab	courseware/labs-source/lab-07-instrumentation-monitoring-kql-feedback.md

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



Visual workflow map for Lab 07.

What you will produce: Telemetry → KQL → Dashboard → Feedback. Keep screenshots, links or generated artifacts as evidence.

Objectives

- Design monitoring strategy.
- Review Azure Monitor and Application Insights.
- Write KQL queries.
- Create feedback loops.

Step-by-Step Guide

93. Identify telemetry required for the sample application.
94. Define application, infrastructure, pipeline, security, and operations metrics.
95. Review Application Insights concepts.
96. Review Azure Monitor Logs concepts.
97. Create or design alert rules.
98. Review pipeline run history and failure rate.
99. Write simple KQL queries.
100. Identify distributed tracing concepts.
101. Define notifications to Teams, GitHub Issues, or Azure Boards.
102. Write an instrumentation strategy note.

Validation

- Metrics connect to operational decisions.
- Alerts have clear action.
- KQL queries are documented.
- Feedback loop is practical.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Evidence checkpoint: Metrics connect to operational decisions.

Review Questions

103. What is MTTR?
104. Why collect pipeline metrics?
105. What is KQL used for?
106. How does feedback improve delivery?

Learner Evidence

- Screenshot or link to the completed work item, branch, pull request, pipeline run, deployment, dashboard or capstone evidence as relevant.

Lab 08 - AZ-400 Capstone: End-to-End DevOps Solution

Source Lab	courseware/labs-source/lab-08-capstone-az400-end-to-end-devops.md



Visual workflow map for Lab 08.

What you will produce: Plan → Code → Build → Release → Monitor. Keep screenshots, links or generated artifacts as evidence.

Objectives

- Design a complete DevOps solution.
- Combine work tracking, source control, pipelines, security, and monitoring.
- Make tradeoff decisions.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Build an exam study plan.

Step-by-Step Guide

107. Review the trainer-provided scenario.
108. Design flow of work using Boards or Issues.
109. Choose a branching and PR strategy.
110. Design CI pipeline stages.
111. Design deployment strategy and environment gates.
112. Add IaC and configuration management.
113. Add secrets, security scanning, and compliance evidence.
114. Add monitoring, alerts, and KQL.
115. Identify risks and mitigations.
116. Create an executive summary.
117. Create technical design notes.
118. Map weak areas to AZ-400 domains.
119. Write a 14-day study plan.

Validation

- Solution includes traceability.
- Pipeline strategy covers build, test, release, and rollback.
- Security and monitoring are built in.
- Study plan targets weak domains.

Evidence checkpoint: Solution includes traceability.

Review Questions

120. Which design tradeoff was hardest?
121. How does the solution improve deployment confidence?
122. What evidence would an auditor need?
123. Which AZ-400 domain needs more review?

Learner Evidence

- Screenshot or link to the completed work item, branch, pull request, pipeline run, deployment, dashboard or capstone evidence as relevant.

Troubleshooting

- Confirm the correct organization, project, repository, branch and service connection are selected.
- Check permissions before creating pipelines, service connections, policies or Azure resources.
- Re-run failed pipelines after reading the first failing task log.
- Never paste secrets into Markdown, pipeline YAML, issue comments or screenshots.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg